

Entrega #1 – Grupo #10

1. Análisis para cada entidad.

- Usuario

Dependencias funcionales (DF)	Forma normal (1FN, 2FN, 3FN, FNBC)	Conclusiones
Documento_ID -> Nombre, Apellido, Universidad, Programa, Correo, Numero celular	Cumple todas las formas.	Solo se tiene a Documento_ID como PK, por lo que no se aplican problemas a solucionar en las formas normales.

- Carros

Dependencias funcionales (DF)	Forma normal (1FN, 2FN, 3FN, FNBC)	Conclusiones
Placa -> Marca, Modelo, Año, Color, Cupos, Usuario_Documento_ID	Cumple todas las formas.	Solo se tiene a Placa como PK, por lo que no se aplican problemas a solucionar en las formas normales.

- Viajes

Dependencias funcionales (DF)	Forma normal (1FN, 2FN, 3FN, FNBC)	Conclusiones
ID_viaje -> Fecha_hora_salida, Direccion_origen, Cupos_disponibles, Costo, Distancia_estimada, Duracion_estimada, Ruta_general, Carros_placa, Localidades ID localidad	Cumple todas las formas.	Cumple todas las formas. Aunque se tenga como FK a Carros_placa y Localidades_ID_localidad, no se repiten datos y se evitan dependencias transitivas.

- Localidades

Dependencias funcionales (DF)	Forma normal (1FN, 2FN, 3FN, FNBC)	Conclusiones
ID_localidad -> Nombre_localidad	Cumple todas las formas.	Cumple todas las formas, pues se tienen dos columnas únicamente, por lo que el único atributo solo depende de la PK.

- Zonas_parada

Dependencias funcionales (DF)	Forma normal (1FN, 2FN, 3FN, FNBC)	Conclusiones
ID_zona_parada -> Nombre_zona, Localidades ID localidad	Cumple todas las formas.	Cumple todas las formas. Los atributos dependen solo de la PK.

- Paradas

Dependencias funcionales (DF)	Forma normal (1FN, 2FN, 3FN, FNBC)	Conclusiones
ID_parada -> Direccion, Zonas_parada_ID_zona_parada	Cumple todas las formas.	Cumple todas las formas. Los atributos dependen solo de la PK.

- Paradas_viaje

Dependencias funcionales (DF)	Forma normal (1FN, 2FN, 3FN, FNBC)	Conclusiones
Viajes_ID_viaje, Paradas_ID_parada -> Numero_parada	Cumple todas las formas.	Se cumple 2FN porque Numero_parada depende solo de la llave candidata PK y no de una sola.

- Solicitudes

Dependencias funcionales (DF)	Forma normal (1FN, 2FN, 3FN, FNBC)	Conclusiones
ID_solicitud -> Fecha_solicitud, Usuario_Documento_ID, Paradas_viaje_Viajes_ID_viaje, Paradas_viaje Paradas ID parada	Cumple todas las formas.	Cumple todas las formas. Los atributos dependen solo de la PK.

- Estados_solicitud

Dependencias funcionales (DF)	Forma normal (1FN, 2FN, 3FN, FNBC)	Conclusiones
ID_estado_solicitud -> Nombre_solicitud	Cumple todas las formas.	Cumple todas las formas, pues se tienen dos columnas únicamente, por lo que el único atributo solo depende de la PK.

- Estados_de_solicitud

Dependencias funcionales (DF)	Forma normal (1FN, 2FN, 3FN, FNBC)	Conclusiones
Solicitudes_ID_solicitud, Estados_solicitud_ID_estado_solicitud -> Fecha_estado	Cumple todas las formas.	Cumple 2FN porque Fecha_estado depende de la llave candidata PK, por lo que cumple 3FN y FNBC. No repetidos en solicitud y estado.

- Estados_viaje

Dependencias funcionales (DF)	Forma normal (1FN, 2FN, 3FN, FNBC)	Conclusiones
ID_estado -> Nombre_estado	Cumple todas las formas.	Cumple todas las formas, pues se tienen dos columnas únicamente, por lo que el único atributo solo depende de la PK.

- Estado_viaje

Dependencias funcionales (DF)	Forma normal (1FN, 2FN, 3FN, FNBC)	Conclusiones
Viajes_ID_viaje, Estados_viaje_ID_estado -> Fecha_estado, Descripcion	Cumple todas las formas.	Cumple 2FN porque Fecha_estado y Descripcion depende de la llave candidata PK, por lo que cumple 3FN y FNBC. No repetidos en viaje y estado.

2. Pruebas de unicidad

Pruebas de unicidad de tuplas (PK)

Si se intenta registrar un conductor con el documento 123456789, el resultado esperado es que se registre sin problema ya que es un documento nuevo, sin embargo, cuando otro usuario intente registrarse con el mismo documento, la base debería fallar porque el documento es la PK de la tabla Usuario y no debería en teoría repetirse.

Pruebas de integridad con FK

Si se intenta registrar un carro con placa ABC123 asociado a un usuario que, si existe en la base, por ejemplo, el documento 123456789, el resultado esperado es que se registre sin problema ya que la FK si encuentra a quien hace referencia. Ahora, si se intenta registrar otro carro, pero con un documento que no está registrado, como 999999999, en este caso la base debería fallar porque esa FK no encuentra ningún usuario al cual hacer referencia.

Pruebas de restricciones de chequeo

Si se intenta crear un viaje con un costo negativo, por ejemplo -20000, el sistema debería fallar ya que el costo de un viaje no puede ser negativo. Lo mismo pasaría si se intentara crear un viaje con cupos disponibles en 0 o en negativo, este tampoco debería dejarse pasar.

RF1 - Registrar conductor

Si un estudiante se registra como conductor con todos sus datos completos (nombre, apellidos, universidad, programa, correo, celular y documento), el registro debería quedar guardado sin ningún problema. En cambio, si intenta registrarse dejando algún campo obligatorio vacío, como la universidad o el documento, la base debería rechazarlo.

RF2 - Registrar vehículo

Si se registra un carro con una placa que todavía no existe en la base, debería quedar registrado normal. Pero si se intenta registrar un carro con una placa que ya está usada por otro vehículo, debería fallar porque la placa es la PK de Carros.

RF3 - Publicar viaje

Si el conductor publica un viaje con fecha, hora, punto de origen, paradas y tarifa, y además lo asocia a un vehículo ya registrado, el viaje debería quedar publicado sin problema. Si intenta publicar un viaje sin asociarle ningún vehículo, esto debería fallar porque un viaje necesita un carro asociado.

RF4 - Gestionar solicitudes

Si el conductor acepta una solicitud de un viaje que todavía está en estado programado, debería aceptarse sin problema y quedar registrada la parada. Ahora, si el conductor intenta aceptar una solicitud, pero el viaje ya paso a estado en curso, no debería dejarlo, ya que un viaje en curso no admite reservas nuevas.

RF5 - Actualizar estado del viaje

Si un viaje pasa de estado programado a en curso, esto debería funcionar sin problema porque es una transición valida. Sin embargo, si se intenta pasar un viaje que ya está completado de vuelta a programado, debería fallar, ya que no se puede modificar el estado de un viaje que ya termino. También debería fallar si se intenta cancelar un viaje que sale en, por ejemplo, una hora, porque no se cumple el mínimo de dos horas de anticipación que pide el negocio.

RF6 - Consultar historial de viajes

Si un conductor consulta su historial, debería poder ver tanto sus viajes pasados como los futuros, junto con el estado de cada uno y los pasajeros que quedaron confirmados en cada viaje.

RF7 - Creación cuenta pasajero

Si un estudiante se registra como pasajero con todos sus datos, debería quedar registrado sin problema. Si intenta registrarse sin poner su correo institucional, la base debería rechazar el registro diciéndole que necesita un correo institucional.

RF8 - Gestión cuenta pasajero

Si un pasajero ya registrado actualiza, por ejemplo, su número de celular, el cambio debería guardarse sin ningún inconveniente.

RF9 - Consultar viajes disponibles

Si alguien busca viajes disponibles filtrando por fecha y zona sin haber iniciado sesión, de todas formas, debería poder ver los resultados, ya que este RF no exige estar autenticado.

RF10 - Solicitar cupo en un viaje

Si un pasajero solicita un cupo y elige un punto de recogida que si esta entre las paradas ofrecidas para ese viaje, la solicitud debería crearse sin problema. Pero si intenta solicitar un punto de recogida que no es parada de ese viaje en específico, debería fallar.

RF11 - Cancelar solicitud

Si un pasajero cancela su solicitud con más de dos horas de anticipación respecto a la salida, debería poder cancelarla sin problema. Si intenta cancelarla con menos de dos horas, la base debería impedirlo.

RFC1 - Consultar demanda por localidad

Si se consultan las solicitudes de cupo agrupadas por localidad dentro de un rango de fechas específico, el sistema debería devolver los conteos correctos para cada localidad dentro de ese rango.

RFC2 - Consultar ingresos por localidad

Si se consultan los ingresos totales generados por la plataforma agrupados por localidad y en un rango de fechas dado, el sistema debería mostrar el total correcto para cada localidad.

RFC3 - Consultar viajes por conductor a través del tiempo

Si se consulta la evolución de los viajes de un conductor en específico, filtrando por localidad y por un rango de fechas, el sistema debería mostrar esa evolución de forma correcta.

RFA1 - Tablero de control del conductor

Si un conductor abre su tablero de control, debería poder ver el total de pasajeros que ha transportado, el tiempo total en viajes, los ingresos generados y la distancia total recorrida.

RFA2 - Tablero de control del pasajero

Si un pasajero abre su tablero de control, debería poder ver cuántos viajes ha tomado, la zona en la que más ha pedido viajes, el total gastado y el promedio de distancia y tiempo por viaje.